RetinAnalysis Setup Guide

Windows, Mac, and Linux — find your section below and follow the steps in order. Skip any step where you already have that tool installed.

Windows

> **Everything below runs in Anaconda PowerShell Prompt**, not Command Prompt and not Git Bash. Windows usually installs two similar-looking shortcuts — **Anaconda Prompt** (Command Prompt) and **Anaconda Powershell Prompt** (actual PowerShell) — you want the second one. If you have Miniconda, look for **Anaconda Powershell Prompt (miniconda3)**. You're in the right one if the prompt starts with `PS`, e.g. `(retinanalysis) PS C:\...>`.

1. Install Git

Download and run: https://git-scm.com/download/win — accept the defaults. (This also installs Git Bash — you won't need it; `git` works fine from Anaconda PowerShell Prompt after this.)

2. Install Docker Desktop

Download and run: https://docs.docker.com/desktop/ — launch it once, then leave it running whenever you need database access. Just `import retinanalysis` doesn't need it running.

3. Install a C++ compiler

Download: https://visualstudio.microsoft.com/visual-cpp-build-tools/ — in the installer, check **"Desktop development with C++"**, then install. (Needed to build one submodule, `vision-utils`.)

4. Confirm you have conda

```
conda --version
```

If that fails, install Miniconda first: https://docs.conda.io/en/latest/miniconda.html

5. Clone the repo

```
git clone https://github.com/yasmine-tani/retinanalysis.git --recursive
cd retinanalysis
```

6. Create the environment and install

```
conda create -y -n retinanalysis python=3.11.13
conda activate retinanalysis
pip install -e .
pip install .\lib\artificial-retina-software-pipeline\utilities\
```

7. Fix a known Windows bug

The first time this package loads `matplotlib`, Windows can raise a DLL load error. If you hit that:

```
pip uninstall Pillow
pip install -U Pillow
```

If you don't hit the error, skip this.

8. Set up config.ini

This file tells the package where your data lives on this machine — it doesn't exist yet, so you're creating it fresh. From the same PowerShell window (already in the `retinanalysis` folder from step 5):

```
notepad src\retinanalysis\config\config.ini
```

Windows will ask "Cannot find config.ini. Do you want to create a new file?" — click **Yes**. Paste in the block below, edit the `[WINDOWS_DEFAULT]` paths, then save (Ctrl+S) and close.

> If that doesn't open a blank file: open Notepad manually, paste the block below, then File → Save As, set "Save as type" to **All Files** (not "Text Documents" — avoids saving as `config.ini.txt`), navigate to `src\retinanalysis\config`, and name it exactly `config.ini`.

```
[WINDOWS_DEFAULT]
analysis = B:\Array-data\sorted
data = B:\Array-data\sorted
raw = B:\Array-data\raw
h5 = B:\Array-data\h5
meta = B:\Array-data\dj_meta
tags = B:\Array-data\tags
query = B:\Array-data\sorted
user = your_username

[WINDOWS_SECONDARY]
analysis = C:\path\to\secondary\analysis
data = C:\path\to\secondary\sorted
raw = C:\path\to\secondary\raw
h5 = C:\path\to\secondary\h5
meta = C:\path\to\secondary\meta
tags = C:\path\to\secondary\tags
query = C:\path\to\secondary\query
user = your_username
```

`B:` is whatever drive letter your shared network drive is mapped to — check File Explorer → This PC. Both sections must be present, but only `[WINDOWS_DEFAULT]` needs real paths — leave `[WINDOWS_SECONDARY]` as placeholder unless you have a second data location. `query` is used by mosaic-plotting across many datasets — point it at the NAS analysis dir even when your other paths point at a local SSD. `config.ini` is gitignored, so `git pull` never touches it.

9. Start the local database

Docker Desktop must be open for this.

```
mkdir ..\retinanalysis-database
copy docker-compose.yaml ..\retinanalysis-database\
cd ..\retinanalysis-database
docker compose up -d
cd ..\retinanalysis
```

10. Install Jupyter and launch it

```
pip install jupyterlab
jupyter lab
```

11. Populate the database

Open any notebook in `demos/` and run in a cell:

```
import retinanalysis as ra
ra.populate_database()
```

Windows troubleshooting

Can't find "Anaconda PowerShell Prompt", or `'x' is not recognized as an internal or external command, operable program or batch file`

You're likely in the cmd-based Anaconda Prompt instead of the PowerShell one — see the note at the top of this section. If no PowerShell-flavored shortcut exists at all, open regular Windows PowerShell, run `conda init powershell` once, then close and reopen PowerShell.

`ModuleNotFoundError: No module named 'cv2'` on the first `import retinanalysis` cell

Not a missing install — `opencv-python-headless` installs automatically with step 6. Jupyter is running a different Python than the `retinanalysis` environment. Close Jupyter, `conda activate retinanalysis`, relaunch `jupyter lab` from that terminal, and check the kernel name top-right — switch it if it's not `retinanalysis` (or `pip install ipykernel` then `python -m ipykernel install --user --name retinanalysis` if it's missing).

`ImportError: DLL load failed while importing cv2`

Rare now with `opencv-python-headless`, but if it happens: confirm only one opencv package is installed (`pip list | Select-String opencv`), try a clean reinstall (`pip uninstall opencv-python-headless -y` , `pip cache purge` , `pip install opencv-python-headless --no-cache-dir`), and check if this is a Windows "N" edition (run `winver`) — N editions lack the Media Feature Pack that video-reading code needs.

`DLL load failed` on `import matplotlib` — see step 7.

`FileNotFoundError: No config file found at ...`

Step 8 was skipped, or the file isn't at exactly `src\retinanalysis\config\config.ini` (check File Explorer isn't hiding the real extension). Redo step 8, then restart the kernel.

`No NAS or SSD paths found, check that one of them is connected` printed but no crash

`config.ini` exists but still has placeholder paths. Confirm `[WINDOWS_DEFAULT]` points at a drive/folder that's really there.

Submodule install fails with a compiler error — the C++ Build Tools from step 3 aren't installed, or open a brand-new PowerShell window after installing them.

Database calls fail / connection refused — Docker Desktop isn't open, or the container isn't running (see the screenshot at the end of this guide).

Cloned without `--recursive` — run `git submodule update --init --recursive`, then redo step 6.

Mac

Everything below runs in **Terminal**.

1. Install Git and a C compiler

```
xcode-select --install
```

This installs both Git and `clang` (the compiler needed later) in one step.

2. Install Docker Desktop

Download the build matching your chip (Apple Silicon or Intel): https://docs.docker.com/desktop/ — launch it once, leave it running whenever you need database access.

3. Confirm you have conda

```
conda --version
```

If that fails, install Miniconda first: https://docs.conda.io/en/latest/miniconda.html

4. Clone the repo

```
git clone https://github.com/yasmine-tani/retinanalysis.git --recursive
cd retinanalysis
```

5. Create the environment and install

```
conda create -y -n retinanalysis python=3.11.13
conda activate retinanalysis
pip install -e .
pip install lib/artificial-retina-software-pipeline/utilities/
```

6. Set up config.ini

This file tells the package where your data lives on this machine — it doesn't exist yet, so you're creating it fresh. From the same Terminal window:

```
nano src/retinanalysis/config/config.ini
```

Paste in the block below, edit the `[DEFAULT]` paths, then save and exit (Ctrl+O, Enter, Ctrl+X).

> If `nano` isn't available: open TextEdit, switch to plain text first (Format → Make Plain Text — saving from Rich Text can corrupt the file), paste the block below, then save as exactly `src/retinanalysis/config/config.ini`.

```
[DEFAULT]
analysis = /Volumes/Array-data/sorted
data = /Volumes/Array-data/sorted
raw = /Volumes/Array-data/raw
h5 = /Volumes/Array-data/h5
meta = /Volumes/Array-data/dj_meta
tags = /Volumes/Array-data/tags
query = /Volumes/Array-data/sorted
user = your_username

[SECONDARY]
analysis = /path/to/secondary/analysis
data = /path/to/secondary/sorted
raw = /path/to/secondary/raw
h5 = /path/to/secondary/h5
meta = /path/to/secondary/meta
tags = /path/to/secondary/tags
query = /path/to/secondary/query
user = your_username
```

`/Volumes/Array-data` is wherever your shared network drive is mounted — check Finder → Go → Network. Both sections must be present, but only `[DEFAULT]` needs real paths. `query` is used by mosaic-plotting across many datasets — point it at the NAS analysis dir even when your other paths point at a local SSD. `config.ini` is gitignored.

7. Start the local database

```
mkdir -p ../retinanalysis-database
cp docker-compose.yaml ../retinanalysis-database/
cd ../retinanalysis-database
docker compose up -d
cd ../retinanalysis
```

8. Install Jupyter and launch it

```
pip install jupyterlab
jupyter lab
```

9. Populate the database

```
import retinanalysis as ra
ra.populate_database()
```

Mac troubleshooting

`ModuleNotFoundError: No module named 'cv2'`

Not a missing install — installs automatically with step 5. Jupyter is running a different Python than `retinanalysis`. Close Jupyter, `conda activate retinanalysis`, relaunch `jupyter lab` from that terminal, check the kernel name top-right.

Submodule install fails with a compiler error — `xcode-select --install` from step 1 didn't finish. Run it again.

`FileNotFoundError: No config file found at ...` — step 6 was skipped or misnamed. Redo it, restart the kernel.

`No NAS or SSD paths found, check that one of them is connected` — placeholder paths still in `config.ini`. Check Finder → Go → Network for the right mount name.

Database calls fail / connection refused — Docker Desktop isn't open or the container isn't running.

Cloned without `--recursive` — run `git submodule update --init --recursive`, redo step 5.

Linux

Everything below runs in a Terminal. Commands assume Debian/Ubuntu (`apt`) — swap in `dnf` / `pacman` /etc. for other distros.

1. Install Git and build tools

```
sudo apt update && sudo apt install -y git build-essential
```

2. Install Docker Engine

Follow: https://docs.docker.com/engine/install/ — then let your user run Docker without `sudo`:

```
sudo usermod -aG docker $USER
```

Log out and back in for that to take effect.

3. Confirm you have conda

```
conda --version
```

If that fails, install Miniconda first: https://docs.conda.io/en/latest/miniconda.html

4. Clone the repo

```
git clone https://github.com/yasmine-tani/retinanalysis.git --recursive
cd retinanalysis
```

5. Create the environment and install

```
conda create -y -n retinanalysis python=3.11.13
conda activate retinanalysis
pip install -e .
pip install lib/artificial-retina-software-pipeline/utilities/
```

6. Set up config.ini

```
nano src/retinanalysis/config/config.ini
```

Paste in the block below, edit the `[LINUX_DEFAULT]` paths, then save and exit (Ctrl+O, Enter, Ctrl+X).

```
[LINUX_DEFAULT]
analysis = /mnt/Array-data/sorted
data = /mnt/Array-data/sorted
raw = /mnt/Array-data/raw
h5 = /mnt/Array-data/h5
meta = /mnt/Array-data/dj_meta
tags = /mnt/Array-data/tags
query = /mnt/Array-data/sorted
user = your_username

[LINUX_SECONDARY]
analysis = /path/to/secondary/analysis
data = /path/to/secondary/sorted
raw = /path/to/secondary/raw
h5 = /path/to/secondary/h5
meta = /path/to/secondary/meta
tags = /path/to/secondary/tags
query = /path/to/secondary/query
user = your_username
```

`/mnt/Array-data` is wherever your shared network drive is mounted. Both sections must be present, but only `[LINUX_DEFAULT]` needs real paths. `query` is used by mosaic-plotting across many datasets — point it at the NAS analysis dir even when your other paths point at a local SSD. `config.ini` is gitignored.

7. Start the local database

```
mkdir -p ../retinanalysis-database
cp docker-compose.yaml ../retinanalysis-database/
cd ../retinanalysis-database
docker compose up -d
cd ../retinanalysis
```

8. Install Jupyter and launch it

```
pip install jupyterlab
jupyter lab
```

9. Populate the database

```
import retinanalysis as ra
ra.populate_database()
```

Linux troubleshooting

`ModuleNotFoundError: No module named 'cv2'` — Jupyter is running a different Python than `retinanalysis`. Close Jupyter, `conda activate retinanalysis`, relaunch `jupyter lab` from that terminal.

`permission denied while trying to connect to the Docker daemon socket` — your user isn't in the `docker` group yet, or you haven't logged out/in since step 2. Run `groups` to check; redo the `usermod` command and fully log out/in if needed.

`FileNotFoundError: No config file found at ...` — step 6 was skipped or misnamed. Redo it, restart the kernel.

`No NAS or SSD paths found, check that one of them is connected` — placeholder paths still in `config.ini`. `df -h` or `mount` can help confirm what's really mounted.

Database calls fail / connection refused — check `docker compose ps` from the `retinanalysis-database` folder, or `docker ps` generally.

Cloned without `--recursive` — run `git submodule update --init --recursive`, redo step 5.

Checking the database container is running

The local database needs its container running before any database-backed call (queries, `ra.populate_database()`) will work. `import retinanalysis as ra` on its own does not need it running.

In Docker Desktop (Windows/Mac), open the app and look at the `retinanalysis-database` container: a stop icon means it's running, a play icon means it's stopped. (See the README on GitHub for a screenshot of what this looks like.)

On Linux (Docker Engine, no GUI), run `docker compose ps` from inside the `retinanalysis-database` folder instead.

Running RetinAnalysis day to day

Once setup is done, starting a normal session is just:

```
conda activate retinanalysis
cd retinanalysis
jupyter lab
```

Getting updates:

```
git pull
pip install -e .   # only needed if dependencies changed
```

Advanced: using uv instead of conda

uv is a fast, Rust-based alternative to conda. It works at the project level — the environment lives in a `.venv` folder inside the repo. Use this instead of the conda steps above if you already know uv and prefer it; the rest of setup (Docker, config.ini, populating the database) is identical either way.

```
git clone https://github.com/yasmine-tani/retinanalysis.git --recursive
cd retinanalysis
uv venv --python 3.11.13
source .venv/bin/activate   # Windows: .venv\Scripts\activate
uv pip install -e .
uv pip install lib/artificial-retina-software-pipeline/utilities/   # Windows: .\lib\artificia
```

Then continue from the config.ini step for your OS above.

Shortcut: install.sh

`install.sh`, included in this repo, bundles several of the manual steps above (create the environment, install retinanalysis and the submodule, create a placeholder `config.ini` if one doesn't exist) into a single command. It's a bash script, so it works out of the box on Mac and Linux Terminal; on Windows it needs Git Bash or WSL, which is why the Windows steps above don't use it.

```
./install.sh conda            # or: ./install.sh uv
./install.sh conda --dev --env retinanalysis   # optional flags
./install.sh uv --python 3.11.13
```

It does not run `git submodule update`, and does not create, populate, or migrate the database — you still need the `git clone --recursive` and Docker/populate steps above. If something fails partway through, the manual steps make it easier to see exactly which command broke, which is why they're the default recommendation.

Upgrading from an older version (DataJoint 2 migration)

Retinanalysis now uses `datajoint==2.2.2`. If you have an existing install from before this change: reinstall or update retinanalysis in your analysis environment, create a fresh local DataJoint/MySQL database (use the Docker steps above), and repopulate it (`ra.populate_database()`) after retinanalysis has been updated. Do not try to update an old DataJoint 0.14 database in place.

We recommend doing this in a fresh conda or uv environment, and keeping the old database and retinanalysis installation around until you've confirmed the updated version isn't causing issues in your analysis code.

Optional: datajoint.json

Retinanalysis provides fallback local DataJoint settings for the common lab workflow, but DataJoint 2 will warn you if it doesn't find a `datajoint.json` file in your project root. You can safely ignore this warning, but if you want to make the connection explicit and avoid it, create a `datajoint.json` file in the root of your analysis project:

```
{
  "database.host": "127.0.0.1",
  "database.port": 3306,
  "database.user": "root",
  "database.password": "simple"
}
```

Project-level `datajoint.json`, environment variables, or explicit `datajoint.config` settings take precedence over retinanalysis' local fallback settings.